



黔南民族师范学院
Qiannan Normal University for Nationalities

本科毕业设计

(终稿)

题目	基于 STM32 嵌入式技术的智能药盒设计与实现		
二级学院	计算机与信息学院		
姓名	朱进	学号	22080900048
专业	软件工程	届别	2026 届
校内指导教师	张娅妮	职称	教授
校外指导教师	李明轩	职称	工程师

二〇二六年五月十四日

黔南民族师范学院本科毕业论文(设计)原创性声明

本人郑重声明：所呈交的本科毕业论文(设计)，是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议，除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

设计作者(签名): 朱进

2026年5月14日

黔南民族师范学院毕业论文（设计）使用授权声明

本人完全了解黔南民族师范学院关于收集、保存、使用毕业论文（设计）的规定，即：

- 按照学校要求提交毕业论文（设计）的印刷本和电子版本；
- 学校有权收藏、使用并向国家有关部门或机构送交论文的印刷本和电子版本；
- 允许论文被查阅和借阅；
- 学校可以公布论文的全部或部分内容，可以采用影印、缩印或其它复制手段保存论文。

作者签名：朱进

指导教师签名：张娅辉

日期：2026 年 5 月 14 日

目录

摘要	1
Abstract	2
1. 引言	3
1.1 选题背景及意义	3
1.2 国内外研究现状	3
1.3 研究的主要内容	4
1.4 论文结构安排	5
2. 系统总体结构	6
2.1 系统设计目标	6
2.2 系统总体架构	6
2.3 硬件平台选型	8
2.4 软件开发环境与工具	11
3. 硬件系统设计	12
3.1 核心控制模块	12
3.2 电源电路设计	12
3.3 传感器模块电路	12
3.4 执行机构电路	13
3.5 通信模块电路	14
3.6 系统整体硬件连接图	14
4. 软件系统设计	15
4.1 软件分层架构设计	15
4.2 主程序流程设计	16

4.3 系统底层模块设计	17
4.4 硬件驱动模块设计	18
4.4.1 蜂鸣器驱动	18
4.4.2 舵机驱动	18
4.4.3 DHT11 温湿度传感器驱动	18
4.4.4 HX711 重量传感器驱动	19
4.4.5 串口驱动	19
4.4.6 ESP8266 与 MQTT 驱动	19
4.5 MQTT 与 WiFi 通信软件设计	19
4.5.1 MQTT 协议简介	19
4.5.2 WiFi 模块工作流程	20
4.5.3 MQTT 消息格式	22
4.5.4 指令解析流程	22
4.6 核心业务逻辑实现	22
4.6.1 服药定时提醒逻辑	22
4.6.2 温湿度采集逻辑	24
4.6.3 药品重量检测逻辑	25
4.6.4 远程设置服药时间逻辑	26
4.7 Android 远程控制客户端核心代码实现	28
4.7.1 MQTT 客户端初始化与连接管理	28
4.7.2 传感器数据解析与界面更新	29
4.7.3 指令下发与远程控制	30
4.7.4 网络监听与系统资源管理	32
5. 系统调试与功能测试	34

5.1 编译与下载调试	34
5.2 单功能模块测试	34
5.2.1 蜂鸣器提醒测试	34
5.2.2 舵机出药测试	34
5.2.3 温湿度采集测试	34
5.2.4 重量传感器测试	35
5.2.5 MQTT 远程通信测试	35
5.3 系统整体联调测试	35
5.4 测试结果分析	36
6. 总结与展望	37
6.1 工作总结	37
6.2 未来展望	37
参考文献	39
致谢	40

摘要

当前我国人口老龄化进程不断加快，老年人口数量持续增长，老年人日常用药管理难题与慢性病群体用药不便的现实困境，已成为社会重点关注的民生问题。目前市场上常见的普通药盒功能单一，只能起到简单的存放作用，而不能实现定时提醒、环境监测、剩余量检测以及远程交互等功能。而在实际应用中容易造成老年人忘记或者服错药的问题，同时无法对药品所处环境有效控制，从而降低药品质量以及用药安全性。

为了更好地应对居家用药管理所面临的问题，本文提出基于 STM32 的嵌入式智能药盒的设计与实现方案。本系统采用 STM32F407 微控制器为主控芯片，连接各种外围设备来完成各项操作。温湿度传感器用来获取药品存放处温度以及湿度信息，保证药品保存在良好环境中；重量传感器可以准确测量药品剩余量，提醒使用者及时补药；WiFi 通信模块可利用 MQTT 进行网络上传输及控制，使家人或者是医生可以在外面就能知道病人服药状况并且给出建议；舵机与蜂鸣器协同工作，可完成自动取药以及服药提醒，在设定好服药时间之后，蜂鸣器会发出提醒声，舵机带动药盒推出对应药物。

本系统集成服药定时提醒、自动出药、温湿度采集、药品余量监测及远程数据上传下达于一体，实现便捷、安全的家庭用药管理系统。经过实验证明，该系统运行良好、稳定，可以大大降低老年人忘记吃药或者吃错药的情况发生，使老人按时吃药成为习惯，保障了他们的身体健康，对家庭用药管理有积极意义，有较好的发展前景和应用前景。

关键词：STM32F407；智能药盒；嵌入式系统；MQTT 通信；传感器监测

Abstract

As population aging accelerates in China, the elderly population keeps rising. Medication management difficulties among the elderly and chronic patients have drawn great social attention. Traditional medicine boxes only support basic drug storage without timing reminder, environmental monitoring, surplus drug detection and remote interaction functions. They often cause missed or mistaken medication, fail to control drug storage conditions, and further affect drug quality and medication safety.

In order to resolve these problems, this paper proposes an STM32-based embedded smart medicine box system. The STM32F407 microcontroller is used as the central device and it implements several features through external modules: a temperature-humidity sensor provides the right conditions of drug storage, measuring the ambient environment in real time; a weight sensor indicates the amount of remaining drugs and prompts refilling; a WiFi module allows remote transmission/control of data with the help of MQTT to allow family or medical personnel to check the state of the medications; a servo motor and buzzer work together to deliver medicine automatically and send timely reminders- when the set time is reached, the buzzer will ring and the motor will release the drugs.

Combining timed reminders, automated dispensing, environmental monitoring, inventory detection and remote control, the system can offer a full smart home medication management solution. The tests prove its stable and reliable work that successfully minimizes the rate of missing/wrong medications and enhances the quality of medications and their safety. It has good technical support and good application prospects and promotion.

Keywords: STM32F407; Smart pillbox; Embedded system; MQTT Communication; Sensor monitoring;

1. 引言

1.1 选题背景及意义

根据国家统计局数据显示，我国进入老龄化社会速度不断加快，老年人口规模不断增大。同时老年人是慢性病高发群体，需长期规律服药以达到治疗目的，保持身体健康。但是在生活中有很多老人由于记忆力减退、思维能力下降以及生活事务繁杂等原因造成漏服药、服错药甚至重复服药现象^[3]。这都会影响药物作用发挥，严重的还会给老年人带来伤害。

药品存放环境会对药效以及用药安全性造成影响。如果温度与湿度过高，则会导致药品受潮，使其中的有效物质逐渐发生分解而降低药效乃至失效等问题。目前市面上大多数药盒只是起到“收纳”的作用，不具备环境监控等功能，不能够知晓药品是否一直处于良好的保存环境中，在老年人独自居住的情况下问题更为突出：没有人能时刻观察他们是否按时吃药，是否已经吃过药，服药过程中缺乏良好的监督及反馈机制，从而增加了他们在家中服用药物的风险。总的来看，传统的药盒功能较为简单，智能化程度不高，已不能适应现代家庭对于健康管理的需求^[12]。

在该背景下，将嵌入式技术、物联网通信技术和传感器检测技术结合起来，开发一款具有智能化、自动化以及远程监控功能的智能药盒，有一定的研究意义以及应用价值。（1）系统可以设置定期提醒以及自动发药的功能，提高患者的用药依从率，降低患者漏服或者错服的概率；（2）对药品储存环境进行时时采集并监测，保证药品在规定时间内处于良好保存状态；（3）利用网络通信功能，使家人或医生能够远程观察患者服药情况^[8]；（4）整个系统设计简单合理，硬件费用低而且易于上手，在一般家庭中普及较为方便。（5）对嵌入式及物联网技术在家用健康监护中应用起到一定指导作用，同时对于类似智能家居产品设计开发也具有参考价值。

1.2 国内外研究现状

国外智能药盒发展较早，在欧美以及日本等地区已经有较为成熟的产品。但是总体而言，大多数产品都可以做到按时提醒、服药记录上传及统计，而一些高端机型还会增加语音识别、GPS 定位等功能。但是放在中国，这样的设备也存在很多问题：首先就是价格较高，其次功能设计过于繁琐，很多功能对中国普通家庭来说并不是必须，

最后是与中国用户的使用习惯以及当地医疗服务体系不匹配。因此，国外相关产品在中国的大范围普及仍然有较大困难。

在我国，随着物联网、嵌入式技术的发展，近年来有关智能药盒的研究及应用也越来越多。但是从现有产品来看，大多数方案仅能做到“按时发出声音或灯光提示”，功能较为简单^[5]。而且一些设备虽然具备蓝牙通信的能力，但是只能用于短距离通信，在温湿度检测、剩余药量判断、基于 MQTT 协议进行远端操作等方面都还有欠缺^[2]。同时由于部分方案受到硬件条件的限制，计算能力和并发性较差，通信方式单一，因此在远程传输稳定性以及多个传感器之间的协调工作上都有所不足^[4]。

综上所述，目前市面上智能药盒的功能性、智能化程度以及远程连接稳定性还有待提高，因此开发一个具有多种传感器监测、自动化操作以及支持 MQTT 远程通讯的智能药盒项目具有实际意义并有研究意义^[11]。

1.3 研究的主要内容

本课题基于 STM32F407 微处理器进行智能化药盒的设计及其实现工作，主要工作有以下几点：（1）系统整体方案设计，确定硬件结构以及软件结构，提出功能要求和目标；（2）硬件电路设计及元器件选择，如主控芯片、温湿度传感器、重量传感器、Wi-Fi 通信模块、舵机、蜂鸣器等硬件之间的连接关系及其驱动等；（3）编写程序代码，采用分层方式编写程序底层驱动、外设驱动、通信协议和业务逻辑部分，例如时钟设置、延时函数、中断服务程序、传感器读取、舵机操作、蜂鸣器报警、MQTT 通信消息接收发送等。（4）具有双向远程通信能力，利用 Wi-Fi 模块以及 MQTT 协议使手机 APP 可以向单片机发送命令并接收其反馈，可远程设置服药时间、随时了解当前环境温湿度及剩余药品数量等^[5]；（5）实现主要功能，即定时服药提醒、舵机自动出药、温湿度实时获取、药品重量及数量监测以及数据上传等；（6）进行程序编译、烧录、调试及功能测试，检查系统的稳定性和正确性并对测试情况进行总结。

1.4 论文结构安排

全文共分为六部分，具体如下：第一章是引言，在这部分中介绍了论文的研究目的和意义，总结了国内外关于该方面的研究成果，同时阐述了本论文的研究内容及章节安排；第二章是系统总体方案的设计，主要包括系统的总体设计目标、系统总体结构设计、硬件选择和开发平台的选择等；第三章是系统的硬件设计，主要是从主控单元、电源模块、各种传感器、执行器以及通信模块这几个方面来介绍各个器件之间的连接关系；第四章是系统的软件设计，主要是从系统软件分层结构、主程序流程、底层驱动程序、通信协议以及系统的主要功能等方面来进行讲解；第五章是对整个系统进行调试以及功能测试工作，在这部分中对编译下载、单个模块的测试、整个系统的联合调试等工作进行了详细的描述，并对测试的结果进行了分析。第六章为总结与展望，在总结本文所做工作的基础上指出现有工作不足以及后续改进方向。

2. 系统总体结构

2.1 系统设计目标

本设计基于对智能药盒的实际应用需求，提出如下几点设计要求：（1）具有定时服药提醒功能，在用户设定的时间节点上，系统会利用蜂鸣器发出警报声以提醒用户按时吃药，同时警报持续时间要符合用户的实际需要，以免造成用户忘记吃药的情况发生^[1]。（2）具有自动发药的功能，用舵机充当执行机构，由舵机正转或者反转来代替药盒盖子的开闭，保证发药流畅，无卡药、漏药现象。（3）具有对药盒内的温度和湿度进行监测并显示的功能，在合适的位置安装合适的温湿度传感器用来检测盒内情况并在显示屏上显示，供使用者查看药品存放条件是否满足要求^[12]。（4）实现药品重量采集及余量显示，在盒子内部安装重量传感器来检测盒内药品剩余重量并将获取到的数据转化为直观余量信息，告知用户及时补充药品；（5）实现 WiFi 联网，利用 MQTT 协议连接到用户的手机 APP，保证消息传递可靠并且低延迟；（6）可以在手机 APP 上设定服药时间，同时将温度、湿度以及药品余量等相关信息发送到手机 APP 进行查看^[3]；（7）保证整个设备正常工作并且快速响应，尽量减少能耗延长使用寿命，简化操作方便老年人和其他人群使用。（8）软件采用分层架构设计，统一编码规范，使代码层次分明、条理清晰、易于后期维护以及增加新功能等。

2.2 系统总体架构

本系统利用三层物联网体系结构，包括感知层、控制层以及应用层，在软件上也采取分层的设计方式。

硬件架构：（1）感知层：采用 DHT11 温湿度传感器、HX711 重量传感器进行环境以及药品信息获取；（2）控制层：基于 STM32F407 作为主控芯片完成对信息分析、决策、外围设备操作及通信任务；（3）执行层：由 SG90 舵机、蜂鸣器构成，在服药时发出提示并且可以实现自动化给药；（4）通信层：使用 ESP8266 WiFi 模块进行 MQTT 通讯；（5）应用层：手机 APP 用于远程操作显示界面等功能。

软件架构：系统软件分为三层：（1）用户应用层（USER）：主程序、业务逻辑以及中断处理函数；（2）系统底层（SYSTEM）：系统初始化、延时函数、时钟配置、中断管理；（3）硬件驱动层（HARDWARE）：传感器、舵机、蜂鸣器、wifi、串口等相关驱动代码。

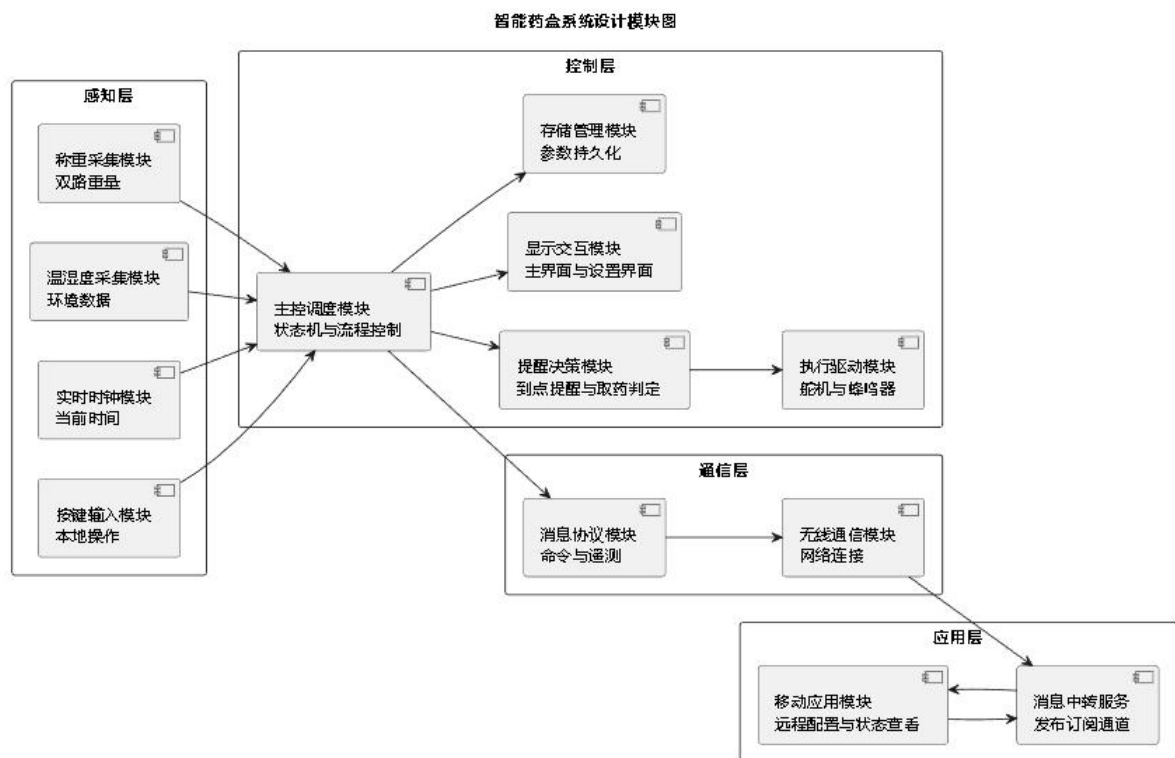


图 2-1 系统架构图

2.3 硬件平台选型

本设计在硬件部分的选择中，在保证性能的基础上兼顾成本以及实用性等，所选用的元器件有以下几种：

(1) 核心控制器：采用意法半导体生产的 STM32F407ZGT6 单片机。该芯片是基于 ARM Cortex-M4 内核的处理器，最大运行速度为 168MHz，片上拥有 1M 字节的 Flash 程序存储器以及 192KB 的 RAM 数据存储器，而且具有众多外围设备可供使用，例如串行端口、定时器、ADC 等，完全可以满足系统的多种传感器的数据采集及多个执行机构协调工作的需求。

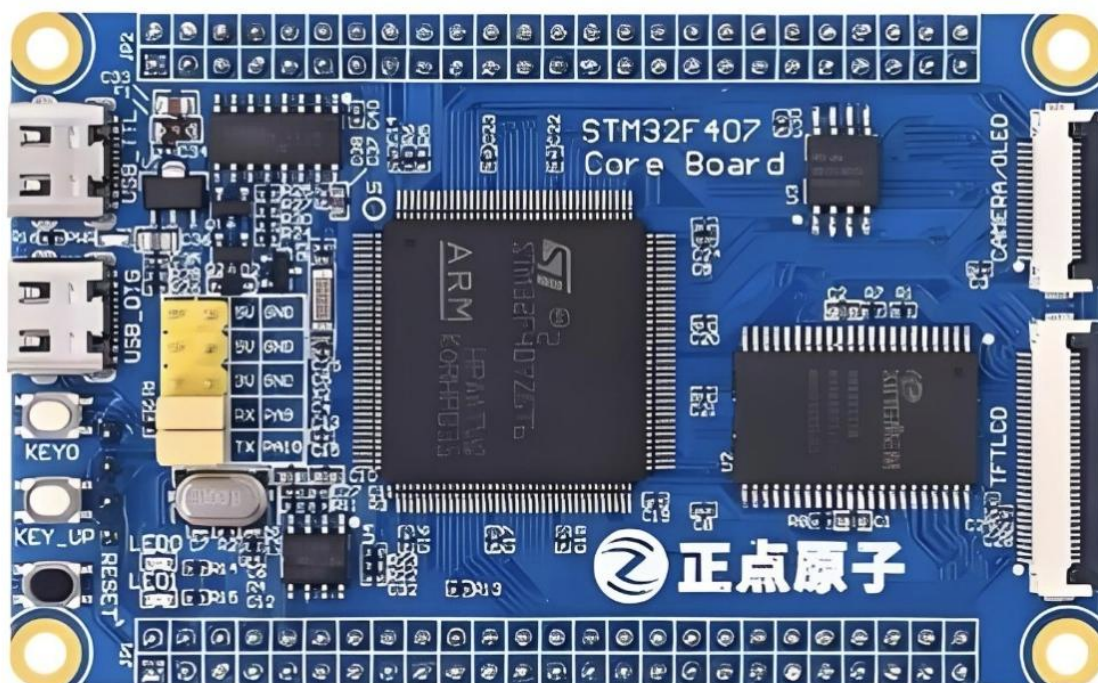


图 2-2 STM32F407 开发板

(2) 温湿度传感器：采用 DHT11 数字温湿度传感器。它可以实现对环境中的温度以及湿度进行测量，使用一条线进行通信，连线简单方便，价格便宜，可以用于室内的药品存放环境检测。

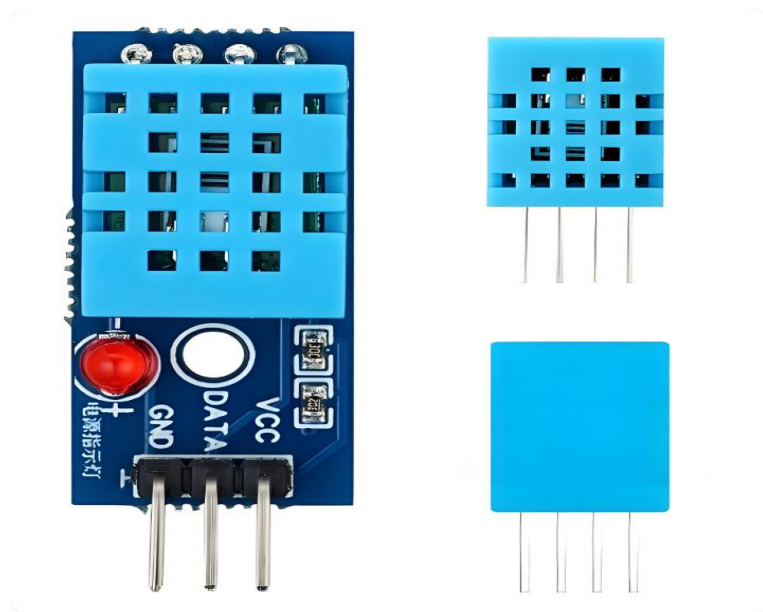


图 2-3 DHT11 数字温湿度传感器

(3) 重量检测部分：采用 HX711+压力传感器方式。HX711 是 24 位高精度 ADC 芯片，与压力传感器结合可以获取重量信息，进而监测药品重量变化情况以及根据该信息得出药品剩余量。



图 2-4 HX711 压力传感器

4) WiFi 通信模块：采用 ESP8266 串口 WiFi 模块。此模块价格便宜，有串口透传及 MQTT 通信功能，方便实现设备联网，实现终端到手机 APP 的数据发送以及接收

控制命令。



图 2-5 ESP8266WiFi 模块

(5) 执行机构采用 SG90 舵机，该部分可以做到精确的角度控制，利用 PWM 信号改变转动的角度从而达到模拟药盒开合以及自动发药的目的，反应迅速且容易操作。



图 2-6 SG90 舵机

(6) 提醒模块：采用有源蜂鸣器作为提醒部件，利用单片机产生的高、低电平来驱动蜂鸣器工作或者停止工作而达到预定的定时提醒的目的，在家庭环境中，这个蜂鸣器发出的声音大小合适，声音清晰，可以满足家庭环境的要求，而且安装简单方便，易于操作，符合本设计的目标。

(7)电源模块:为了使系统的各个部分都能良好地运行,本次设计使用一个5V/2A电源对整个电路供电,在此电源下可以给系统内的单片机、舵机、WiFi模块和其他各种各样的传感器等提供稳定的电源,防止由于电源问题而引起的硬件损坏或者是系统无法正常使用的情况发生,使整个系统能持续不断地运行,达到用户的使用要求。

2.4 软件开发环境与工具

本设计主要使用C语言进行编程,在具体开发过程中所用到的相关软件及设备有:

(1) 操作系统为 Windows11; (2) 编译器为 Keil uVision5; (3) 程序烧写工具为 ST-LINK 等下载器; (4) 调试工具为串口调试助手以及 MQTT 调试工具; (5) 代码组织形式为 USER、SYSTEM、HARDWARE、OBJ 四层结构。

系统编译完成后会自动生成相应的可执行固件,然后用下载工具下载到 STM32F407 上电即可独立工作,在裸机环境下开发无需加载任何操作系统,具有启动迅速、可靠性和稳定性良好以及对硬件资源要求较低的优点,符合本文的设计要求。

3. 硬件系统设计

3.1 核心控制模块

本系统采用 STM32F407 作为主控芯片进行数据分析、计算等操作并控制各外围设备工作。为了使该芯片可以顺利上电运行，在其基础上设计相应的最小系统，包括电源部分、复位部分、时钟部分和下载口等，这是整个系统的基础。

芯片主频设置为 168MHz，给整个系统带来很强计算力；有多个通用定时器，用来控制舵机 PWM 信号以及延时；有多个串口外设，可以连接到 ESP8266 WiFi 模块和串口调试工具上；还有 GPIO 接口，用于读取传感器信息及控制蜂鸣器^[6]。

在本系统中，STM32F407 主要负责以下几个方面的工作：（1）初始化系统时钟、延时函数以及中断等；（2）读取 DHT11 温湿度传感器及 HX711 重量传感器的数据；（3）产生 PWM 信号以控制舵机转动的角度；（4）驱动蜂鸣器发出服药提醒的声音；（5）利用串口与 ESP8266 进行通信，完成 MQTT 协议下数据的发送与接收；（6）检测是否到服药的时间点，进行定时提醒以及发药操作；（7）向云端发送信息的同时也可以获取云端下发的信息并对相关参数进行设置等。

3.2 电源电路设计

电源模块是整个系统正常运行的基础，在本设计中采用外部连接 5V 电源给系统供电，然后利用稳压电路把电压降至 3.3V 供单片机以及各传感器等器件工作。

电源电路设计如下：（1）输入电压：5V/2A，满足舵机以及 WiFi 模块大电流要求；（2）稳压电路：使用 AMS1117-3.3 稳压器将 5V 转化为 3.3V 给 STM32、DHT11、HX711 供电；（3）滤波电路：在电源输入以及稳压器输出处设置电容滤波电路以降低电压纹波提高供电质量；（4）电源指示：增设一个 LED 电源指示灯以便观察设备是否通电。

STM32F407 单片机是 3.3V 电平设备，不能承受 5V 电平信号，所以传感器以及通信模块都使用 3.3V 供电，防止电平过高造成器件损坏；舵机耗电量较大，从 5V 电源直接受电，以获得足够的转动转动力量以及较快反应速度。

3.3 传感器模块电路

本系统包含两种传感器：温湿度传感器与重量传感器。

(1) DHT11 温湿度传感器电路 DHT11 使用单总线通信方式,只需要一条数据线就可以与单片机 GPIO 相连。

VCC: 接 3.3V 电源;

GND: 接地;

DATA: 接 STM32 通用 GPIO 口,用于数据收发;

增加上拉电阻,保证通信稳定性。

单片机每隔一定时间产生一个触发信号,然后读取 DHT11 返回的数字信号,从而得出当前环境的温度以及湿度。

(2) HX711 重量传感器电路 HX711 是一个 24 位高精度 AD 转换器,用于把压力传感器产生的模拟量转化为数字信号。

VCC: 接 5V 电源;

GND: 接地;

DT 与 SCK: 连接到 STM32 的两个 GPIO 引脚上,用来传输数据以及时钟。

压力传感器电桥与 HX711 模拟输入端连接。

上电后, HX711 不断获取压力传感器数据,单片机从其 GPIO 端读取重量,进行校正计算得到药品真实重量并与其起始重量比较得出药品剩余量。

3.4 执行机构电路

系统执行部分采用舵机以及蜂鸣器进行控制,用于自动上药、定时服药报警等^[9]。

(1) SG90 舵机电路由 PWM 脉冲信号决定角度大小,工作电压为 5V。

VCC: 接 5V 电源;

GND: 接地;

PWM 信号端: 接 STM32 定时器通道引脚。

单片机利用自身定时器产生每隔 20 毫秒一个周期 PWM 信号,改变高电平持续时间实现舵机转动 0 到 180 度,在本项目中,舵机作为药盒开关使用,在需要服药时转动一定角度,在完成发药操作之后会自动回到初始位置。

(2) 蜂鸣器电路采用有源蜂鸣器,由三极管驱动控制。

蜂鸣器正极接 5V, 负极接三极管集电极;

三极管基极通过限流电阻接 STM32GPIO 口;

当 GPIO 输出高电平，三极管导通，蜂鸣器工作；反之，则不工作。

达到设定服药时间后，单片机控制 GPIO 引脚输出高电平，蜂鸣器发出声音，以提醒患者服药。

3.5 通信模块电路

ESP8266WiFi 模块用于给系统进行网络连接，通过串口与 STM32 进行交互。

VCC: 接 3.3V 电源；

GND: 接地；

TXD: 接 STM32USARTRX；

RXD: 接 STM32USARTTX；

使能端与复位端接 3.3V，保证模块正常工作。

单片机利用串口给 ESP8266 发送 AT 命令，进行 WiFi 设置，之后连接相应的 MQTT 服务器，进行消息发布和接收的操作。而手机端可以通过 MQTT 服务器给 ESP8266 发送服药时间等控制命令，ESP8266 接收到命令后传送给单片机；同时单片机会把获取到温度、湿度、药品重量以及服药情况等信息上传到手机上显示，从而达到两者之间互相通信的目的。

3.6 系统整体硬件连接图

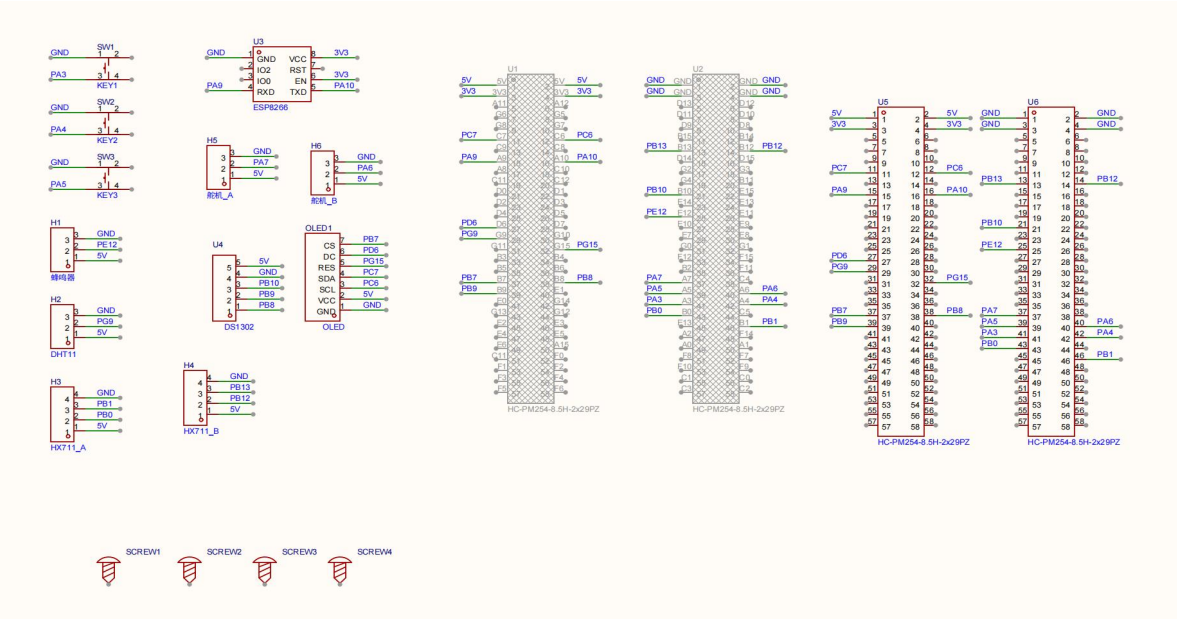


图 3-1 系统整体硬件连接图

4. 软件系统设计

4.1 软件分层架构设计

本系统软件严格采用三层分层架构，与项目目录结构完全对应，分别为用户应用层、系统底层、硬件驱动层。分层设计可降低代码耦合度，提高可读性、可维护性与可移植性。

(1) 用户应用层（USER 目录）作为系统最高层，负责核心业务逻辑实现与任务调度，包含文件：

main.c：系统主函数入口；

main.h：全局宏定义与函数声明；

stm32f4xx_it.c：中断服务函数。

主要功能：系统初始化、循环调度、服药时间计算、命令解析、数据上送及逻辑处理等。

(2) 系统底层（SYSTEM 目录）为整个系统提供基础支撑服务，不依赖硬件业务，包含文件：

sys.c/sys.h：系统时钟初始化、寄存器配置；

delay.c/delay.h：毫秒、微秒精准延时函数；

interrupt.c/interrupt.h：中断分组以及中断使能控制。

主要功能：提供稳定系统时钟、通用延时函数、中断管理等，为上层驱动以及应用程序提供服务。

(3) 硬件驱动层（HARDWARE 目录）负责与硬件外设直接交互，封装底层操作，向上层提供统一调用接口，包含文件：

beep.c/beep.h：蜂鸣器驱动；

servo.c/servo.h：舵机驱动；

dht11.c/dht11.h：温湿度传感器驱动；

hx711.c/hx711.h：重量传感器驱动；

esp8266.c/esp8266.h：WiFi 模块驱动；

mqtt.c/mqtt.h：MQTT 协议解析；

uart.c/uart.h：串口驱动。

主要功能：实现外设初始化、数据读取、指令控制等硬件相关操作。

(4) OBJ 目录编译自动生成目录，存放.o 目标文件与.elf、.bin 可执行固件文件，无需手动修改代码。

4.2 主程序流程设计

main.c 为系统入口函数，程序执行流程如下：

- 1.系统上电启动；
- 2.执行系统初始化：时钟配置、延时函数初始化、中断初始化；
- 3.硬件外设初始化：蜂鸣器、舵机、温湿度传感器、重量传感器、串口、WiFi 模块。
- 4.ESP8266 连接 WiFi 网络，连接 MQTT 服务器，订阅远程控制主题。
- 5.首次采集温湿度、重量数据，上传至 APP；
- 6.进入主循环(while(1))；
- 7.循环采集温湿度数据与药品重量数据；
- 8.判断当前时间是否与预设服药时间匹配；
- 9.若匹配，则启动蜂鸣器提醒，控制舵机转动取出药品，延时后使舵机回到原位，关闭蜂鸣器并且上传服药信息。
- 10.判断是否收到 APP 远程指令；
- 11.若收到指令：解析服药时间，更新系统时间参数；
- 12.定时上传温湿度、重量、余量数据至 APP；
- 13.循环执行上述流程。
- 14.主程序使用轮询方式进行多任务调度，简洁、可靠，在裸机上应用。

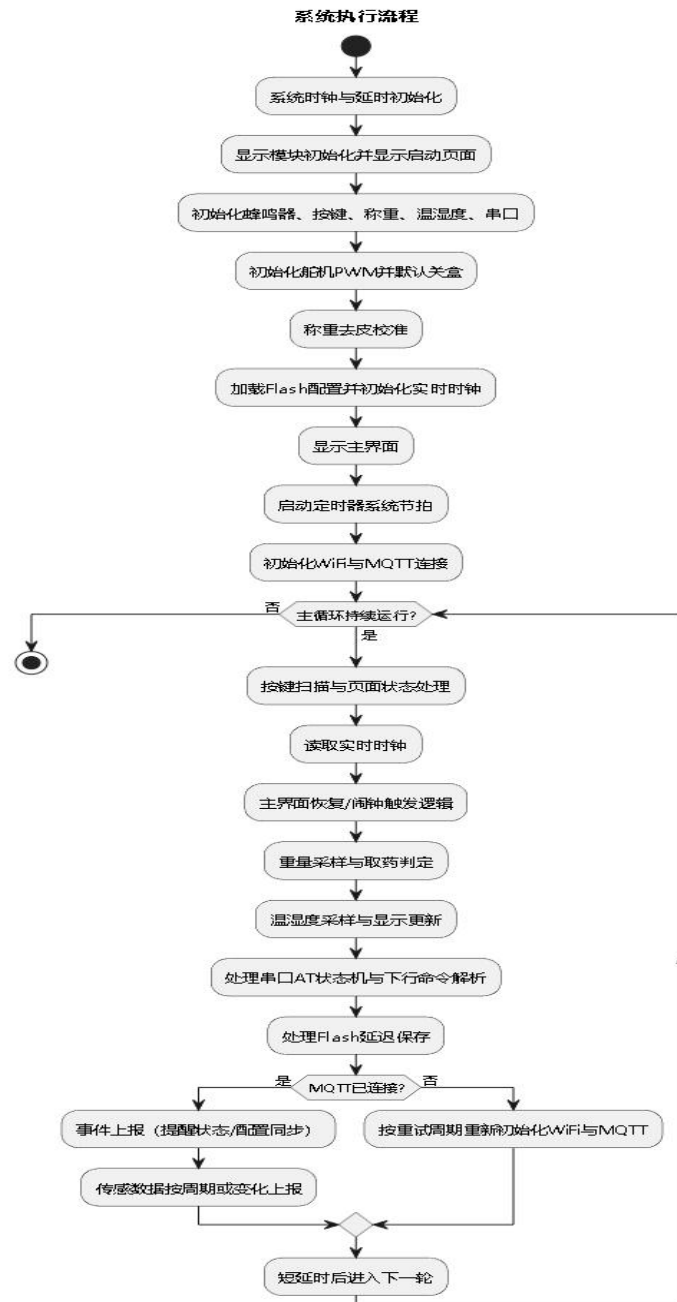


图 4-1 系统执行流程图

4.3 系统底层模块设计

系统底层模块是整个系统的基石，包括时钟、延时、中断等。

(1) 系统时钟配置(sys.c)，STM32F407 外部晶振为 8MHz,经过 PLL 倍频为 168MHz，在该文件中定义了一个函数 `SystemInit(void)`，用于初始化时钟源以及设置倍频系数并开启相应的外设时钟，使系统能够达到最大运行速度。

(2) 延时函数(delay.c)利用系统时钟进行延时，有两个延时函数：

void delay_ms(uint16 nms): 毫秒级延时;

void delay_us (uint32 nus): 微秒级延时.

延时函数是传感器通信、舵机控制、蜂鸣器控制基础，保证时序准确性。

(3) 中断管理 (interrupt.c) 函数: Interrupt_Init()功能: 初始化中断组、中断优先级，对定时器中断以及串口中断进行相关配置等。

系统底层模块对上层隐藏寄存器操作，给上层提供简单易用的函数接口，提高开发效率。

4.4 硬件驱动模块设计

4.4.1 蜂鸣器驱动

驱动文件: beep.c、beep.h 核心函数:

BEEP_Init(void): 初始化 GPIO 为推挽输出模式。

BEEP_On(void): 开启蜂鸣器;

BEEP_Off(void): 关闭蜂鸣器。

工作过程: 当服药时间到，主程序执行 BEEP_On(), 蜂鸣器响一声，然后用户拿药，再执行 BEEP_Off()结束提醒。

4.4.2 舵机驱动

驱动文件: servo.c、servo.h 舵机控制基于定时器 PWM 输出进行，可控制的角度范围为 0-180 度。主要函数有:

SERVO_Init(void): 初始化定时器及 PWM 输出。

SERVO_SetAngle(uint8_tangle): 设置舵机转动角度.

工作原理: 系统启动时舵机处于 0 度闭合状态，要投放药品时，设置角度为 90 度以模拟药盒打开，等待几秒钟后回到初始位置即 0 度，完成投放药品的过程。

4.4.3 DHT11 温湿度传感器驱动

驱动文件: dht11.c、dht11.h 核心函数:

DHT11_Init(void): 初始化 GPIO 口;

DHT11_ReadData(uint8 *temp,uint8 humi): 获取温度以及湿度值。

工作原理: 单片机发出起始信号，DHT11 收到此信号后返回 40 位的数据，即湿

度整数、湿度小数、温度整数、温度小数以及一个校验值，程序对其处理得到所需数据。

4.4.4 HX711 重量传感器驱动

驱动文件：hx711.c、hx711.h 核心函数：

HX711_Init(void): 初始化 GPIO 口；

HX711_Read(void): 读取原始 AD 数据；

Get_Weight(void): 获取校准后实际重量值。

工作原理：压力传感器把重量变成一个模拟信号，然后由 HX711 转化为数字信号，单片机读取此数字信号经过滤波及校正运算，得到药品真实重量以及相对于最初设定重量剩余比例。

4.4.5 串口驱动

驱动文件：uart.c、uart.h 用于与 WiFi 模块、调试工具通信。主要函数有：

UART1_Init(uint32_t bound):设置串口波特率。

UART_SendData(uint8 *buf,uint16 len): 串口发送数据；

USART1_IRQHandler(): 串口接收中断服务函数。

串口接收中断用来及时接收 WiFi 模块下发 MQTT 命令，以实现远程控制及时性。

4.4.6 ESP8266 与 MQTT 驱动

驱动文件：esp8266.c、mqtt.c 核心功能：

ESP8266 发送 AT 指令连接 WiFi；

连接 MQTT 服务器；

订阅主题（接收 APP 指令）；

发布主题（上传温湿度、重量、状态）；

解析 JSON 格式指令，提取服药时间。

通信过程为：WiFi 连接→MQTT 连接→订阅命令主题→不断发送数据→接受及解析命令→做出响应。

4.5 MQTT 与 WiFi 通信软件设计

4.5.1 MQTT 协议简介

MQTT 是轻量级发布/订阅消息协议，适用于物联网设备在低带宽、弱网络情况

下可靠通讯。而在本次开发中，STM32 使用 ESP8266 连接到 MQTT Broker，订阅`home/esp/command`接收来自 APP 下发命令（例如启动/停止提醒、设置闹钟等）以及向`home/esp/telemetry`发送自身状态及传感器信息（温度、湿度、两路重量、提醒状态、配置同步等），进而达到 APP 与药盒之间互动目的。

4.5.2 WiFi 模块工作流程

ESP8266 软件工作流程：

- 1.发送 AT 指令测试模块是否正常；
- 2.设置 WiFi 模块为 STA 模式；
- 3.连接 WiFi 名称与密码；
- 4.获取 IP 地址，联网成功；
- 5.建立 TCP 连接，接入 MQTT 服务器；
- 6.发送 MQTT 连接包，完成服务器连接；
- 7.发送订阅包，订阅远程控制主题；
- 8.保持心跳包，维持 MQTT 连接；
- 9.等待 APP 指令，定时上传数据。

WiFi 模块工作流程图

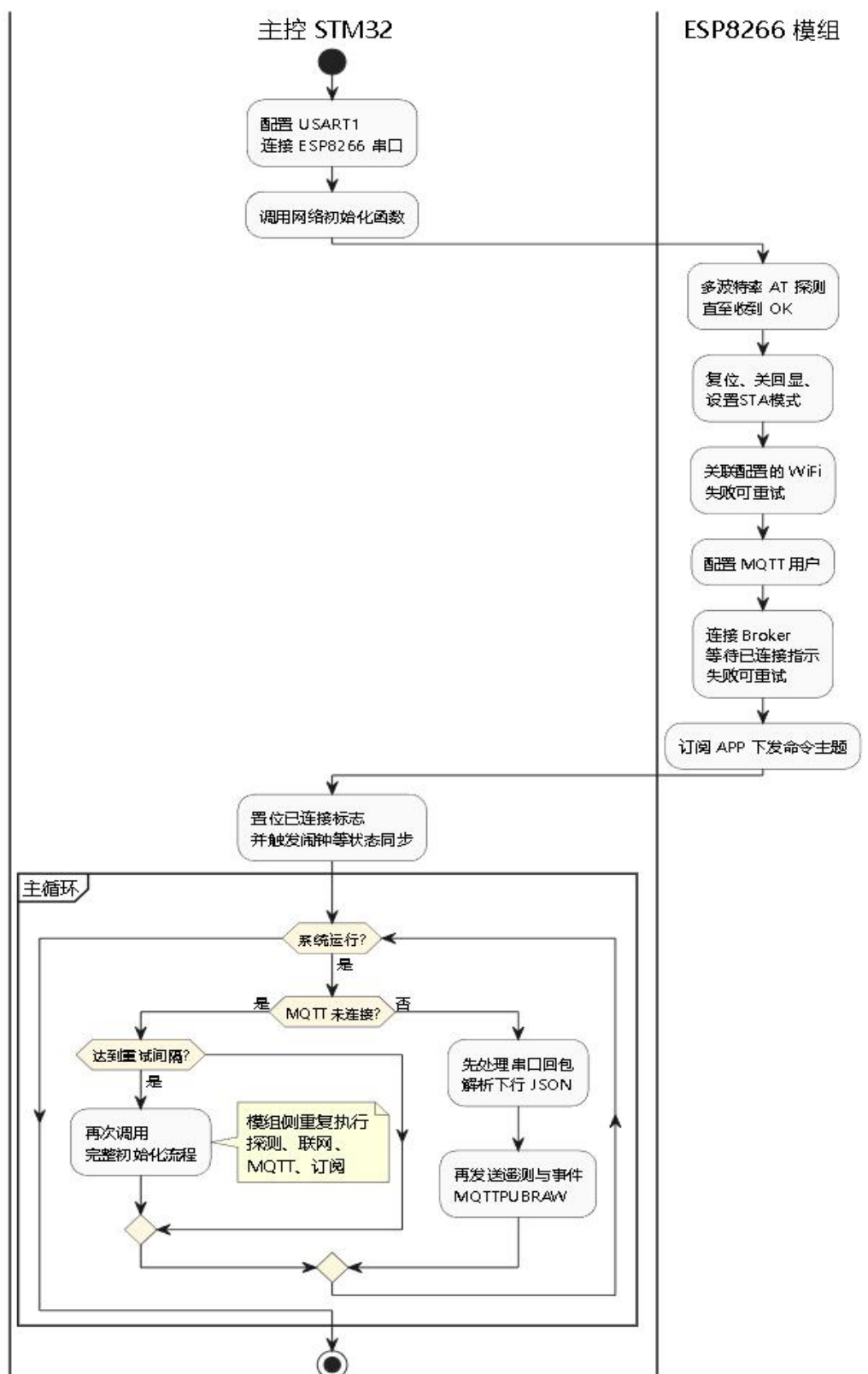


图 4-2 WiFi 模块工作流程图

4.5.3 MQTT 消息格式

(1) 上传数据格式 (JSON) {

"temperature":25,

"humidity":60,

"weight":150,

"remain":80,

time":"08:00"

}

(2) 接受指令格式(JSON){"set_time":"08:00"}

4.5.4 指令解析流程

- 1.串口接收 MQTT 消息;
- 2.提取有效数据段;
- 3.解析 JSON 字符串;
- 4.提取 set_time 字段;
- 5.更新系统服药时间变量;
- 6.保存至内存, 立即生效。

4.6 核心业务逻辑实现

4.6.1 服药定时提醒逻辑

1. 系统中设置好服药时间 (例如: 08:00, 12:00, 18:00 等)。
- 2.实时时钟与服药时间对比;
- 3.时间匹配时进入提醒流程;
- 4.启动蜂鸣器;
- 5.舵机旋转打开药盒;
- 6.延时 5 秒, 等待用户取药;
- 7.舵机复位, 关闭蜂鸣器;
- 8.上传服药成功状态至 APP。

服药定时提醒流程

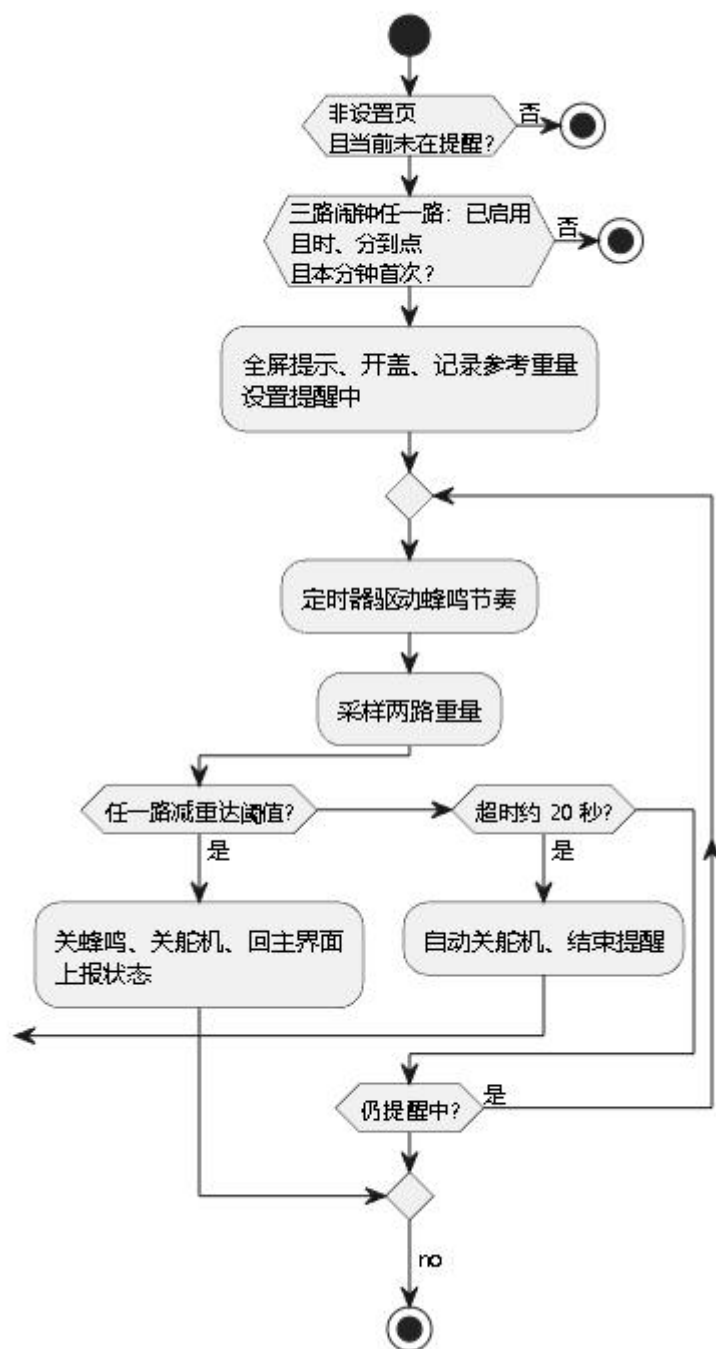


图 4-3 服药定时提醒流程图

4.6.2 温湿度采集逻辑

- 1.每隔 2 秒采集一次 DHT11 数据；
- 2.进行数据滤波，去除异常值；
- 3.保存温度值与湿度值；
- 4.上传至 MQTT 服务器；
- 5.APP 端实时显示。

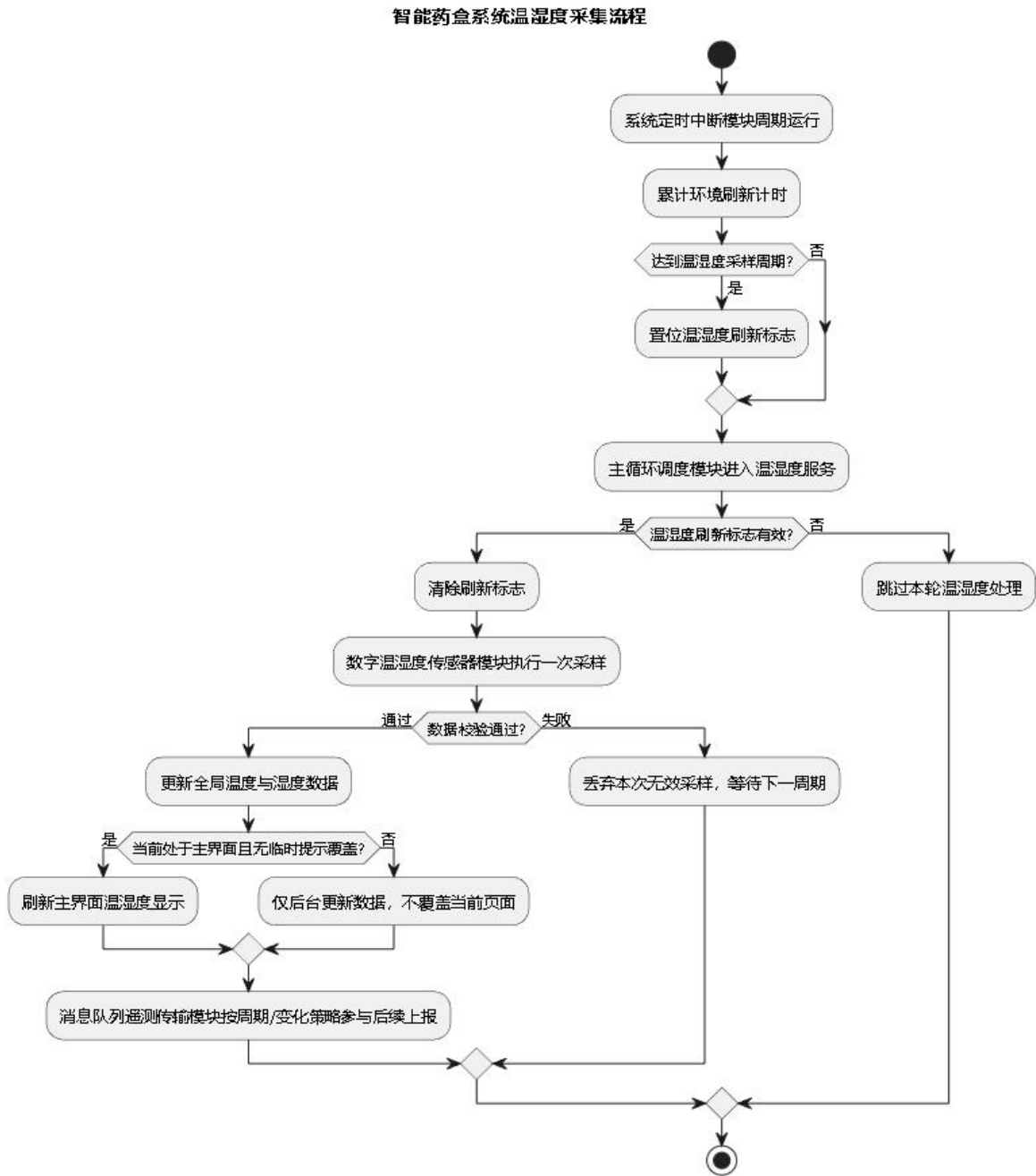


图 4-4 温湿度采集流程图

4.6.3 药品重量检测逻辑

- 1.系统上电采集初始重量；
- 2.定时采集当前重量；
- 3.计算剩余重量
- 4.重量过低时可扩展提醒功能；
- 5.上传余量至 APP。

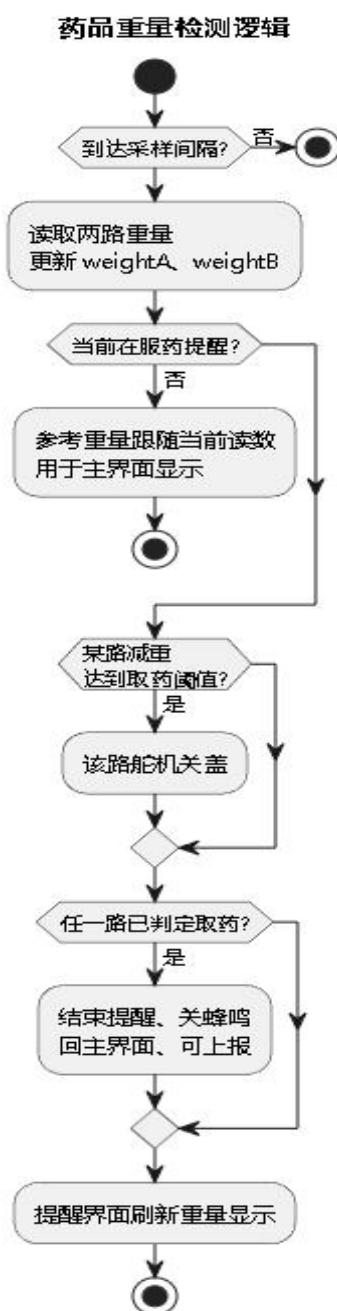


图 4-5 药品重量检测逻辑流程图

4. 6. 4 远程设置服药时间逻辑

- 1.APP 编辑服药时间并下发；
- 2.MQTT 服务器转发消息；
- 3.ESP8266 接收并通过串口发送；
- 4.单片机串口中断接收数据；
- 5.JSON 解析提取时间；
- 6.覆盖原时间参数；
- 7.新提醒规则立即生效。

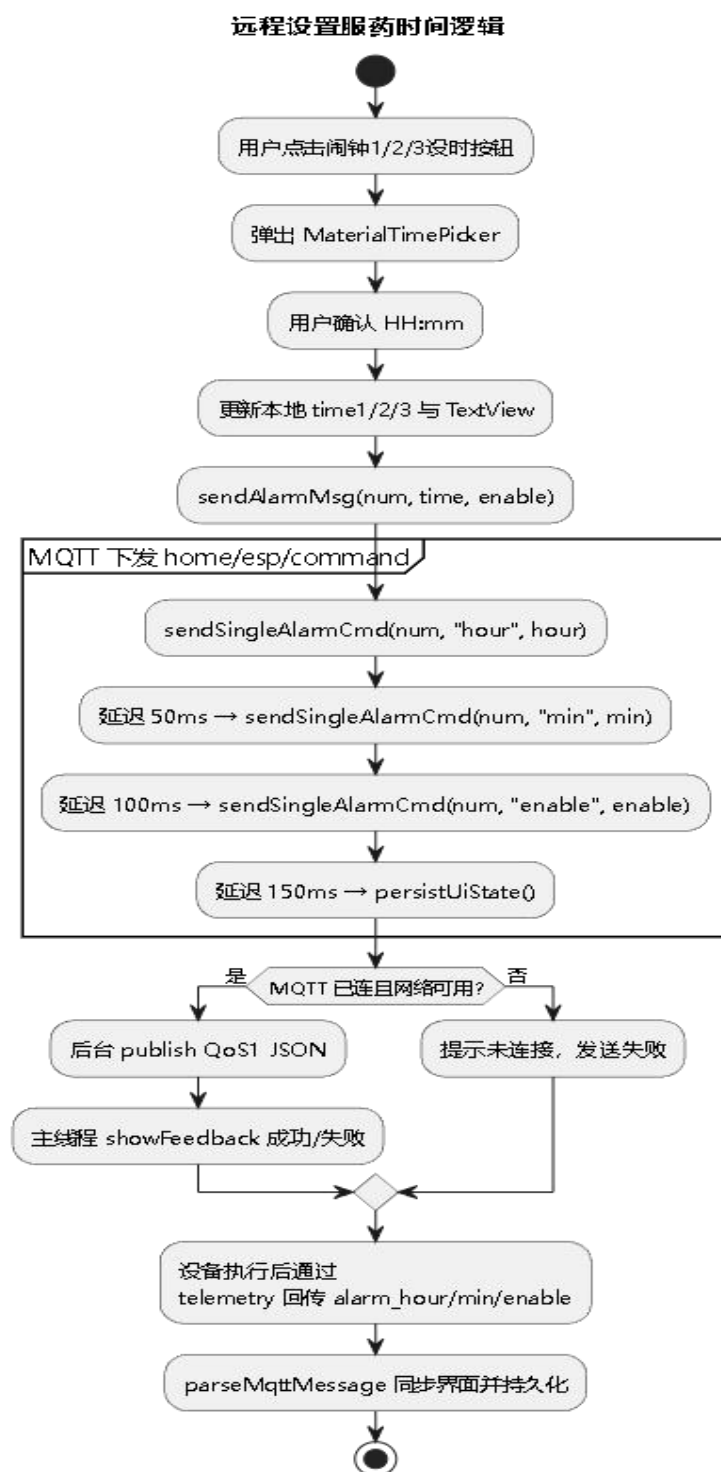


图 4-6 远程设置提醒流程图

4.7 Android 远程控制客户端核心代码实现

为了使智能药盒系统能够实现远程监测以及参数调整，在此使用 Android Studio 编写一款手机端的应用程序，使用 MQTT 轻量级物联网协议并结合 ESP8266 WiFi 模块进行通信。该应用程序完成对 MQTT 客户端的连接管理、解析来自传感器的数据信息、设定服药的时间、发送命令、检测网络状况并且保存本地的状态等操作，给用户提供了一个简单易用的方法来对设备进行远程控制。

4.7.1 MQTT 客户端初始化与连接管理

Android 端使用 Eclipse Paho MQTT 库建立客户端，实现自动重连、消息订阅、心跳保活等功能以保证与智能药盒设备良好通信，在初始化过程中设置 MQTT 服务器地址、客户端 ID、连接超时时间和心跳间隔，连接成功后订阅设备遥测主题。MQTT 初始化以及连接相关代码如下：

```
/**
 * MQTT 客户端初始化配置
 */
private void initMqtt() {
    try {
        // 生成稳定客户端 ID，避免重复连接
        clientId = getOrCreateStableClientId();
        // 创建 MQTT 客户端并配置本地持久化
        client = new MqttClient(host, clientId, new MqttDefaultFilePersistence(
            getDir("mqtt", MODE_PRIVATE).getAbsolutePath()));

        // 设置连接参数
        options = new MqttConnectOptions();
        options.setCleanSession(false);
        options.setConnectionTimeout(15);
        options.setKeepAliveInterval(60);
        options.setAutomaticReconnect(true); // 开启自动重连

        // 设置 MQTT 消息回调
        client.setCallback(new MqttCallbackExtended() {
            // 连接/重连完成，重新订阅数据主题
            @Override
            public void connectComplete(boolean reconnect, String serverURI) {
                try {
                    client.subscribe(TOPIC_TELEMETRY, 1);
```

```

        handler.sendMessage(1); // 通知 UI 连接成功
    } catch (MqttException e) {
        e.printStackTrace();
    }
}

// 连接丢失，触发重连逻辑
@Override
public void connectionLost(Throwable cause) {
    handler.sendMessage(4);
}

// 接收设备上报数据
@Override
public void messageArrived(String topic, MqttMessage message) {
    Message msg = new Message();
    msg.what = 3;
    msg.obj = topic + "---" + new String(message.getPayload());
    handler.sendMessage(msg);
}

});
} catch (Exception e) {
    e.printStackTrace();
    client = null;
}
}

```

4.7.2 传感器数据解析与界面更新

Android 端接收 ESP8266 发送的 JSON 数据，解析温度、湿度、药品重量、闹钟是否开启等信息，同步到界面上并将其保存到本地以防止重新启动时数据丢失，主要的数据解析以及 UI 更新代码为

```

/**
 * 解析智能药盒上传的 JSON 数据，更新界面与本地状态
 */
private void parseMqttMessage(String msg) {
    try {
        String payload = msg.split("---")[1];
        JSONObject json = new JSONObject(payload);

        // 解析温湿度数据
    }
}

```

```

        if (json.has("temp")) {
            lastTemp = json.getInt("temp");
            tv_temperature.setText("温度: " + lastTemp + " °C");
        }
        if (json.has("humi")) {
            lastHumi = json.getInt("humi");
            tv_humidity.setText("湿度: " + lastHumi + "%");
        }

        // 解析药品重量数据
        if (json.has("weightA")) {
            lastWeightA = json.getInt("weightA");
            tv_weight_a.setText("重量 A: " + lastWeightA + " g");
        }

        // 解析闹钟使能状态
        if (json.has("alarm1_enable")) {
            alarm1Enable = json.getInt("alarm1_enable") == 1;
            sw_alarm1.setChecked(alarm1Enable);
        }

        persistUiState(); // 本地持久化保存参数
    } catch (Exception e) {
        showFeedback("数据解析失败", false);
    }
}

```

4.7.3 指令下发与远程控制

APP 利用 MQTT 协议给智能药盒发送命令，例如设置服药时间、是否开启提醒、是否开启闹钟等，命令以 JSON 形式进行打包，在子线程中发送以免阻塞主线程。远程指令下发主要代码为

```

/**
 * 下发控制指令到智能药盒终端
 */
private void publishMqtt(String topic, String msg, String tip) {
    if (!isConnected || !isNetworkAvailable) {
        showFeedback("未连接 MQTT/网络，发送失败", false);
        return;
    }
}

```

```

// 子线程发布消息，防止 UI 阻塞
new Thread() -> {
    try {
        MqttMessage message = new MqttMessage();
        message.setPayload(msg.getBytes());
        message.setQos(1); // 保证消息可靠到达
        client.publish(topic, message);

        Message handlerMsg = new Message();
        handlerMsg.what = 5;
        handlerMsg.obj = tip + "成功";
        handler.sendMessage(handlerMsg);
    } catch (MqttException e) {
        Message handlerMsg = new Message();
        handlerMsg.what = 6;
        handlerMsg.obj = tip + "失败";
        handler.sendMessage(handlerMsg);
    }
}).start();
}

```



图 4-7 APP 远程指令图

4.7.4 网络监听与系统资源管理

为了提高本系统在各种网络环境下稳定运行的能力，APP 实时检测网络状况，在网络中断的情况下停止 MQTT 连接，在网络恢复之后重新建立连接；当程序退出时断开连接并销毁相应的线程以及消息队列等。网络监听以及生命周期的相关代码如下

```
/**
 * 注册系统网络状态监听
 */
private void registerNetworkListener() {
    ConnectivityManager cm = (ConnectivityManager) getSystemService(Context.CONNECTIVITY_SERVICE);
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N && cm != null)
    {
        cm.registerDefaultNetworkCallback(new
ConnectivityManager.NetworkCallback() {
            // 网络恢复，自动重连 MQTT
            @Override
            public void onAvailable(Network network) {
                handler.sendMessage(8);
            }
            // 网络断开，暂停连接
            @Override
            public void onLost(Network network) {
                handler.sendMessage(7);
            }
        });
    }
}

/**
 * 退出应用时释放资源
 */
@Override
protected void onDestroy() {
    super.onDestroy();
    try {
        if (client != null && client.isConnected()) {
            client.disconnect();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```
}  
scheduler.shutdownNow(); // 关闭定时重连任务  
handler.removeCallbacksAndMessages(null);  
}
```

5. 系统调试与功能测试

5.1 编译与下载调试

(1) 点击编译器一键构建，编译器自动编译 USER、SYSTEM、HARDWARE 目录下的.c 文件，生成目标文件存放至 OBJ 目录，最终生成.bin 可执行文件。编译过程无报错、无警告，说明代码语法与配置正确。

(2) 程序下载使用 ST-LINK 等下载器连接单片机与电脑，在下载工具里把.bin 文件写入到 STM32F407 Flash 内。下载完毕后，系统会自动复位并且开始执行，电源指示灯亮起即表示系统工作正常。

(3) 串口调试用串口调试助手观察系统的运行日志，如 WiFi 连接状况、MQTT 连接状况、传感器采集的数据以及命令下发情况等，方便排查故障。

5.2 单功能模块测试

5.2.1 蜂鸣器提醒测试

测试方法：设置服药时间，观察蜂鸣器能否准时报警。测试步骤：

- 1.系统上电；
- 2.设置服药时间为当前时间；
- 3.等待时间匹配；
- 4.蜂鸣器发出提示音，持续 5 秒后停止。

测试结果：蜂鸣器响应及时，提醒声音清晰，功能正常。

5.2.2 舵机出药测试

测试方法：触发提醒功能，观察舵机旋转角度是否合理。测试步骤：

- 1.舵机初始化角度 0 度；
- 2.进入服药流程，舵机旋转至 90 度；
- 3.延时 5 秒后，自动返回 0 度；

测试结果：舵机动作流畅、角度准确，能够有效模拟药盒开关功能。

5.2.3 温湿度采集测试

测试方法：使用标准温度计采集数据与之对比。测试步骤：

- 1.系统上电采集数据；
- 2.串口打印温度、湿度值；

3.与标准设备对比；

测试结果：数据误差在允许范围内，采集稳定可靠。

5.2.4 重量传感器测试

测试方法：使用标准砝码进行校准测试。测试步骤：

1.空载采集零点；

2.放置已知重量砝码；

3.读取显示重量；

测试结果：测量精度高，能够准确反映药品重量与余量。

5.2.5 MQTT 远程通信测试

测试方法：手机 APP 发送命令，检查单片机是否能接收到并且进行相应的操作。

测试过程为：

1.WiFi 联网成功，MQTT 已连接；

2.APP 下发服药时间指令；

3.单片机接收并更新参数；

4.到达时间后执行提醒；

测试结果：通信稳定、指令无丢失、响应延迟低。

5.3 系统整体联调测试

将所有模块组合进行整体系统测试，测试流程如下：

1.系统上电，初始化全部外设；

2.自动连接 WiFi 与 MQTT 服务器；

3.实时显示温湿度、药品重量、药品余量；

4.APP 远程设置服药时间为 19:00；

5.时间到达 19:00，蜂鸣器启动提醒；

6.舵机旋转打开药盒，5 秒后复位；

7.自动上传服药记录至 APP；

8.持续监测环境数据并定时上传。

9.整个系统运行流畅，各个模块之间配合良好，无问题出现，无卡顿、无数据丢失等。

5.4 测试结果分析

通过对系统各模块与整体功能测试，得出以下结论：

(1) 功能实现情况系统全部设计功能均已实现：

定时服药提醒功能正常；

舵机自动出药功能正常；

温湿度采集与显示功能正常；

药品重量检测功能正常；

WiFi+MQTT 远程通信功能正常；

APP 远程设置服药时间功能正常。

表 5-4 测试结果

测试模块	测试内容	预期结果	实际测试结果	测试结论
蜂鸣器提醒	定时到达后声光提醒	准时报警、声音清晰	准时触发、提醒正常	合格
舵机出药	定时到达后自动出药	旋转到位、复位正常	动作流畅、无卡滞	合格
温湿度采集	DHT11 数据读取	数据稳定、误差合理	采集准确、波动小	合格
重量检测	HX711 重量与余量计算	数值准确、可算余量	精度满足使用要求	合格
MQTT 通信	APP 远程设置服药时间	指令下发、终端响应	通信稳定、无丢包	合格
系统整体	多模块协同运行	无死机、无异常重启	连续 24 小时运行稳定	合格

(2) 系统的稳定性系统连续运行 24 小时无死机、无重启、无数据异常，传感器采集稳定，通信连接良好，执行机构动作正确，能够满足长期工作的要求。

(3) 实时性远程指令下发之后反应时间小于 1 秒，服药提醒无延迟，数据上传间隔可以设置，能满足一般需求。

(4) 实用性强的操作简便，用户只需要在手机上操作就可以完成所有设置，十分方便，适合老人、家属以及照护者等使用。

以上就是本设计完成所有要求的功能，系统运行良好、实用性强，符合设计要求。

6. 总结与展望

6.1 工作总结

本次设计完成了基于 STM32 嵌入式技术的智能药盒系统设计与实现。设计围绕服药管理、环境监测与远程控制三大核心需求，依次开展硬件电路设计、软件开发与系统联调等工作，确保系统稳定可靠运行。

硬件设计以 STM32F407 为核心控制器，搭建了完整的硬件平台。系统外设包括 DHT11 温湿度传感器、HX711 重量传感器、ESP8266 WiFi 模块、SG90 舵机及蜂鸣器等模块，可分别完成环境与重量数据采集、执行机构控制及远程通信等任务，为系统功能实现提供硬件基础。

软件设计采用分层架构思想，将系统划分为应用层、驱动层与中间层，使代码结构清晰，便于后期维护与功能扩展。软件开发依次完成系统初始化、外设驱动开发、定时服药提醒、舵机自动出药、MQTT 双向通信及数据解析上传等功能，实现了从底层驱动到上层业务逻辑的完整开发。

经系统全面测试，结果表明本智能药盒系统可稳定实现定时服药提醒、自动出药、温湿度监测、药品余量检测及 APP 远程配置等预设功能。系统运行稳定、响应及时，具有较强实用性。该系统有效解决了传统药盒功能单一、智能化水平低、缺乏远程管理等问题，适用于家庭养老、社区护理及慢性病患者日常服药管理等场景^[10]。

6.2 未来展望

本次设计已完成智能药盒主要功能实现，可以正常使用，但是考虑到实际情况，还有很大提升及发展空间，后续可以从以下几点进行改进：

（1）增加实时时钟模块（RTC），当前系统定时基于单片机自带定时器，存在较大误差，添加 RTC 后可以大大提高定时精度并且可以为每组设定不同的服药时间以满足不同服药需求。

（2）增加语音播报模块，当前服药提醒仅通过蜂鸣器报警，不够直观，在此基础上增加语音播报功能，可直接播报服药时间和药量等信息^[7]，使用户体验更好，尤其是老年人。

（3）增加电池供电及电量检测，目前设备使用外接电源供电，不方便携带，增加可充电电池供电以及电量检测后，可以随身携带并且可以随时查看剩余电量以防突

然没电而不能使用。

(4) 添加数据保存功能，当前系统只能进行实时更新相关数据，不能保存以往服药情况，在此基础上增加该功能可以自动生成每日常服药时间和剩余药量变化等内容，形成完整服药记录便于患者及医生查阅。

(5) 改进 MQTT 通信加密方式，在目前系统中 MQTT 通信未进行加密，有信息被窃取的风险，改善该问题可以提高数据安全，保证用户隐私。

(6) 扩展蓝牙通信功能，当前系统只支持 WIFI 联网通信，在没有网络的情况下不能进行远程设置，添加蓝牙通信之后可以在没有网络的情况下进行较近的距离的设置，提高系统的实用性。

(7) 开发完善的微信小程序以及手机 APP 应用程序，目前的应用程序只能实现基本的控制以及显示功能，在未来可以进一步丰富其内容，添加药品分析、用药建议等功能，以帮助患者更好的管理自己的用药情况。

本次设计实现了一个较为完整的智能药盒系统，在嵌入式技术和物联网技术应用于智能医疗健康的方面提出了一种可行方案，有较大的发展空间以及实用性。后续可以对其进行产品化改进，以解决一些在使用过程中的问题，使该系统能够更好地投入到生活中去，服务于家庭养老、慢病管理等领域。

参考文献

- [1] 郭宇凡,金佳鑫.基于 STM32 单片机的智能药盒设计[J].电子制作,2024,32(19):95-98.
- [2] 周霞.基于窄带物联网技术的智能药盒终端设计与实现[J].自动化与仪器仪表,2024,(07):293-296+301.
- [3] 周婷婷,苏梦婷,田崑颖,陈宇露,顾思佳.基于移动医疗应用技术接受模型的社区老年慢性病患者智能药盒使用体验的质性研究[J].军事护理,2024,41(01):1-4.
- [4] 刘明硕,何丹丹,李帅.基于 STM32 的老年群体居家智能药盒系统设计与实现[J].电子设计工程,2023,31(21):161-165.
- [5] 董鸿宇,李佳睿.基于 STM32 和物联网技术的智能药盒设计[J].物联网技术,2023,13(06):108-110+117.
- [6] 童江,吴志敏,赵海梅.基于嵌入式技术的智能药盒设计[J].电脑与信息技术,2023,31(03):61-64.
- [7] 吴帆,周旭光,吴霖华,黎海凌.基于 STM32 带定位功能的老年人急救智能药盒设计[J].轻工科技,2023,39(03):108-111+182.
- [8] 常国权,尚进辉,孙帅霞.基于物联网的智能药盒设计[J].现代信息科技,2022,6(18):162-165+170.
- [9] 刘玉丛.基于人机交互系统的取药机器人设计[J].电子技术与软件工程,2022,(16):83-86.
- [10] 高笛烟,黄泽豪,陈志聪.面向慢性病患者的智能家居药盒的设计与实现[J].电子技术与软件工程,2022,(14):109-112.
- [11] 吴振磊,顾楚楚.基于 stm32 嵌入式技术的智能药盒设计[J].甘肃科技纵横,2020,49(04):1-3.
- [12] 苏小礼,白洁,葛龙涛,刘瑞凯.基于 STM32 的多模态光电传感老人智能药盒用药安全监测系统[J].家用电器,2025,67(12):6-11.

致谢

时光飞逝，本科四年求学生涯也将要结束，在最后一篇毕业论文完成之时，我要感谢所有在我人生道路上给予关怀、支持以及帮助的老师、同学和家人。

首先，我要向我的导师张娅妮老师表示最诚挚的谢意。从论文题目确定、方案设计、硬件选择、程序编写、系统测试以及论文撰写过程中，张老师对我悉心指导、悉心教导。当我在技术上遇到困难或者论文思路不清楚时，张老师都会及时提醒我注意的问题并给出合理有效的意见和建议，使我能够很好地完成基于 STM32 的智能药盒的设计与实现。张老师的认真负责的工作态度、渊博的知识以及无私奉献的精神对我影响很大，在今后的学习工作中都将是我宝贵的财富。

感谢黔南民族师范学院计算机与信息学院各位教师，在大学四年学习期间，他们教导我掌握物联网以及软件工程相关知识，使我能够自主进行毕业设计。课堂教学内容以及实验过程中帮助都对我此次毕业设计起到良好作用。

感谢我的家人和同学，一直以来都是我的坚强后盾，在学业以及毕业设计过程中，家人的理解和支持还有生活中的关心照顾，使我能够专心致志地完成学业及各项任务。

最后，感谢在此次设计过程中参考的所有学者相关研究工作，为本人提供有益的理论依据和技术指导，同时也感谢对这篇论文进行审阅并提出宝贵意见和建议的所有评委老师，在百忙之中抽出时间阅读这篇文章。

本次毕业设计是我对大学四年专业知识的一次总结与检验，也是对我一次很好的锻炼机会，在以后的工作中我会把学到的知识运用到工作中去，继续保持认真负责的态度，不断学习进步，争取将来在嵌入式及物联网方面能有更多收获来报答学校和社会。